FUNDAMENTOS DE PROGRAMACIÓN

Programar en C desde cero

Ejemplos completos que compilan, errores típicos y ejercicios · explicado paso a paso

1 Introducción · **2** Algoritmos · **3** Tipos de datos
4 Operadores · **5** Leer y escribir · **6** Secuenciales
7 if-else y switch · **8** Bucles
9 Variables locales y globales · **10** Funciones
11 Vectores y matrices · **12** Números aleatorios
13 Punteros · **14** Cadenas · **15** Memoria dinámica
16 Estructuras · **17** Ficheros · **18** Varios archivos
19 Algoritmos clásicos · **20** Depuración y proyecto final
Chuleta · 18 errores típicos · 22 ejercicios



✿ Guía de programación en C · de cero a funciones y vectores

♡ De qué va esta guía (en cristiano)

Tu asignatura **Fundamentos de Programación** tiene este orden de materiales: *01 Introducción → 02 Diseño de algoritmos → 03 Tipos de datos → 04 Operadores → 05 Leer y escribir → 06 Estructuras secuenciales → 07 Selectivas → 08 Iterativas → variables locales y globales → 09 Funciones → 10 Vectores y matrices*, y las prácticas 2–6 (introducción, if-else/switch, bucles, funciones, vectores y números aleatorios).

Esta guía sigue **exactamente ese orden** (capítulos 1–12) y después **completa** lo que necesitas para programar en C con soltura: punteros, cadenas, memoria dinámica, estructuras, ficheros, varios archivos, algoritmos clásicos y depuración (capítulos 13–20). Parte de cero y no da nada por sabido. En cada capítulo tienes: la idea, la sintaxis, **ejemplos completos que puedes copiar y ejecutar** (los he compilado todos para asegurarme de que funcionan) y los errores típicos.

> **Ojo.** Esta guía la he hecho a partir de los títulos de tu asignatura en el aula virtual, no de los PDFs de la profesora (solo he visto las capturas). Cubre todo lo que esos temas necesitan, pero si algún PDF usa otra notación o pide algo concreto, mándamelo y lo ajusto.

Cómo usarla: lee un capítulo, **escribe tú los ejemplos a mano** (no copies y pegues, así se aprende de verdad), cámbiales cosas y mira qué pasa. Programar se aprende programando.

✿ Capítulo 1 · Introducción: tu primer programa

¿Qué es C? Un lenguaje de programación **compilado**: tú escribes un texto (el *código fuente*, archivo `.c`) y un programa llamado **compilador** (`gcc`) lo traduce a un ejecutable que el ordenador entiende.

Tu primer programa:

```
#include <stdio.h>

int main(void)
{
    printf("Hola, mundo!\n");
    return 0;
}
```

Línea por línea:

- `#include <stdio.h>` → "trae la librería de entrada/salida" (la que tiene `printf` y `scanf`). Sin esto no puedes escribir en pantalla.
- `int main(void)` → **todo programa en C empieza por la función `main`**. El ordenador ejecuta lo que hay dentro de las llaves `{ }`, de arriba abajo.
- `printf("...");` → escribe texto en pantalla. `\n` es un **salto de línea**.
- `return 0;` → "el programa terminó bien".
- **Cada instrucción termina en punto y coma `;`.** Olvidarlo es el error número 1.

Cómo compilarlo y ejecutarlo (en la terminal de VS Code / WSL / Ubuntu):

```
gcc hola.c -o hola
./hola
```

- `gcc hola.c -o hola` compila `hola.c` y crea el ejecutable `hola`.
- `./hola` lo ejecuta. (Si usas matemáticas como `sqrt` añade `-lm` al final: `gcc hola.c -o hola -lm`.)
- Consejo: compila siempre con avisos: `gcc -Wall hola.c -o hola`. Te avisa de errores sutiles.

Comentarios (el ordenador los ignora, son notas para humanos):

```
// comentario de una línea
/* comentario
   de varias líneas */
```

Errores típicos de principiante:

- Falta un `;` o una llave `}` → el compilador da error en esa línea o en la siguiente.
- Escribir `Printf` o `PRINTF`: **C distingue mayúsculas de minúsculas**.

- Olvidar `#include <stdio.h>`.
- Leer los errores: el compilador te dice el archivo, la **línea** y qué pasa. Empieza siempre por el **primer** error; los demás suelen ser consecuencia de ese.

✿ Capítulo 2 · Diseño de algoritmos

Antes de escribir código hay que **pensar**. Un **algoritmo** es una lista ordenada de pasos para resolver un problema, sin ambigüedad y que termina.

> **Receta para diseñar un programa:**

1. **Entender el problema:** ¿qué datos entran? ¿qué tiene que salir?
2. **Pensar los pasos** (pseudocódigo, en tu idioma).
3. **Traducir a C.**
4. **Probar** con varios casos (normales y raros: 0, negativos...).

Ejemplo: media de tres notas.

Entrada: tres notas. *Salida:* su media. *Pseudocódigo:*

```
1. Leer nota1, nota2, nota3
2. media = (nota1 + nota2 + nota3) / 3
3. Escribir media
```

Todo algoritmo se construye con solo **tres tipos de piezas**:

- ✿ **Secuencia:** instrucciones una tras otra (Cap. 6).
- ✿ **Selección:** "si pasa esto, haz aquello, si no..." (Cap. 7).
- ✿ **Iteración:** repetir mientras algo se cumpla (Cap. 8).

Ejemplo con selección y repetición — "adivina el número":

```
1. Elegir un número secreto
2. Repetir:
     Pedir un número al usuario
     Si es menor que el secreto → decir "más alto"
     Si es mayor → decir "más bajo"
   hasta que acierte
3. Escribir "¡Acertaste!"
```

Los **diagramas de flujo** dibujan lo mismo con cajas: óvalo = inicio/fin, rectángulo = acción, rombo = pregunta (sí/no), paralelogramo = entrada/salida.

✿ Capítulo 3 · Tipos de datos y variables

Una **variable** es una "cajita con nombre" donde se guarda un dato. En C, **hay que decir de qué tipo es** antes de usarla.

Tipo	Qué guarda	Ejemplo	Se escribe con
`int`	números enteros	`int edad = 18;`	`%d`
`float`	decimales (precisión simple)	`float x = 3.14f;`	`%f`
`double`	decimales (doble precisión, **mejor por defecto**)	`double pi = 3.14159;`	`%f` (printf) / `%lf` (scanf)
`char`	**un** carácter	`char letra = 'A';`	`%c`
`long` / `long long`	enteros más grandes	`long long n = 10000000000LL;`	`%ld` / `%lld`

Declarar, asignar, inicializar:

```
int edad;          // declarar (la caja existe, pero tiene basura dentro)
edad = 18;         // asignar un valor
int curso = 1;     // declarar e inicializar a la vez (¡lo mejor!)
```

Reglas para los nombres: letras, números y `_`; no pueden empezar por número ni tener espacios ni ser palabras reservadas (`int`, `if`, `for`…). Usa nombres claros: `nota_final`, no `x7`.

Constantes (valores que no cambian):

```
#define PI 3.14159      // sin ; al final
const int MAX = 100;    // alternativa moderna
```

Los caracteres son números. `'A'` vale 65 en la tabla ASCII, `'a'` vale 97, `'0'` vale 48. Por eso `'a' + 1` es `'b'`.

Cuidado con la división entera:

```
int a = 7, b = 2;
printf("%d\n", a / b);        // 3  (¡no 3.5! entero entre entero = entero)
printf("%f\n", (double)a / b); // 3.500000  (con cast)
```

`(double)a` es un **cast** (conversión de tipo): convierte `a` a `double` *antes* de dividir.

Programa completo: tamaños de los tipos y desbordamiento.

```c
#include <stdio.h>
#include <limits.h>

int main(void)
{
    printf("int ocupa %zu bytes\n", sizeof(int));
    printf("double ocupa %zu bytes\n", sizeof(double));
    printf("char ocupa %zu bytes\n", sizeof(char));
    printf("El int mas grande: %d\n", INT_MAX);
    return 0;
}
```

- ✿ `sizeof` te dice cuántos bytes ocupa un tipo.
- ✿ Un `int` suele ocupar 4 bytes: llega hasta **2 147 483 647**. Si te pasas, da la vuelta (*overflow*) y salen números negativos absurdos.

Errores típicos: usar una variable sin inicializar (tiene basura); creer que `1/2` es 0,5 (es 0); guardar decimales en un `int` (se **trunca**: `int x = 3.9;` guarda 3).

✿ Capítulo 4 · Operadores

Aritméticos: `+ - * / %`. El `%` **es el resto de la división entera** (solo con enteros):

```
7 / 2    // 3  (cociente)
7 % 2    // 1  (resto)
10 % 5   // 0  → es múltiplo
```

Usos clásicos del `%`: saber si un número es **par** (`n % 2 == 0`), si es **múltiplo** de otro, sacar la **última cifra** (`n % 10`) y quitarla (`n / 10`).

Asignación: `=` guarda el valor de la derecha en la variable de la izquierda. Versiones compactas:

```
x += 5;   // x = x + 5
x -= 2;   // x = x - 2
x *= 3;   // x = x * 3
x /= 4;   // x = x / 4
x++;      // x = x + 1   (incremento)
x--;      // x = x - 1   (decremento)
```

Relacionales (dan 1 si es verdad y 0 si es falso; en C no hay "true/false" básico, **0 = falso, cualquier otra cosa = verdadero**):

`==` igual · `!=` distinto · `<` menor · `>` mayor · `<=` menor o igual · `>=` mayor o igual.

⚠ **El error más famoso de C:** `=` **asigna**, `==` **compara**. Escribir `if (x = 5)` asigna 5 a x y siempre es verdadero. Debes escribir `if (x == 5)`.

Lógicos: `&&` (**y**), `||` (**o**), `!` (**no**).

```
if (edad >= 18 && edad < 65)   // entre 18 y 64
if (letra == 'a' || letra == 'e')
if (!(n > 0))                  // lo contrario de n > 0
```

✿ **No** se puede escribir `0 < x < 10` como en mates. Se escribe `x > 0 && x < 10`.

Precedencia (quién se hace primero): paréntesis → `! ++ --` → `* / %` → `+ -` → `< > <= >=` → `== !=` → `&&` → `||` → `=`. Si dudas, **pon paréntesis**; nunca estorban.

Operador ternario (un if-else en una línea): `condición ? valor_si_verdad : valor_si_falso`.

```
int mayor = (a > b) ? a : b;
```

Ejemplo completo: operadores en acción.

```c
#include <stdio.h>

int main(void)
{
    int a = 17, b = 5;
    printf("%d + %d = %d\n", a, b, a + b);
    printf("%d / %d = %d (cociente)\n", a, b, a / b);
    printf("%d %% %d = %d (resto)\n", a, b, a % b);
    printf("a > b es %d\n", a > b);
    printf("a == b es %d\n", a == b);
    a += 3;
    a++;
    printf("a ahora vale %d\n", a);
    return 0;
}
```

(Para escribir el símbolo `%` en `printf` hay que poner `%%`.)

✿ Capítulo 5 · Leer y escribir (entrada/salida)

Escribir con `printf`. Se escribe un texto con **marcadores** (`%d`, `%f`...) y después, separadas por comas, las variables que ocupan cada hueco, en orden.

Marcador	Para	Ejemplo
`%d`	int	`printf("%d", 5);` → 5
`%f`	float/double	`printf("%f", 2.5);` → 2.500000
`%.2f`	decimal con **2 cifras**	`printf("%.2f", 2.5);` → 2.50
`%c`	un carácter	`printf("%c", 'A');` → A
`%s`	una cadena de texto	`printf("%s", "hola");` → hola
`%5d`	entero en **5 espacios** (alineado)	`printf("%5d", 42);` → `  42`
`\n`	salto de línea	
`\t`	tabulador	

Leer con `scanf`. Lee lo que teclea el usuario y lo guarda **en una variable**. Lleva `&` delante de la variable (significa "la dirección donde guardar"):

```
int edad;
scanf("%d", &edad);        // para int
double x;
scanf("%lf", &x);          // para double: ¡%lf en scanf!
char c;
scanf(" %c", &c);          // para char: espacio antes del %c
```

Reglas de oro del `scanf`:

- **Siempre `&`** delante de la variable (excepto en cadenas de texto).
- `float` → `%f`; `double` → `%lf` (**en scanf**, no `%f`).
- Para leer un carácter, pon un **espacio antes**: `" %c"`. Así ignora el *Enter* que quedó pendiente de la lectura anterior. (Si no, "se salta" la lectura: es un clásico.)
- Siempre escribe un `printf` antes del `scanf` para que el usuario sepa qué teclear.
- Para varias variables: `scanf("%d %d", &a, &b);` (separadas por espacio o Enter).

Programa completo: pedir datos y mostrarlos.

```c
#include <stdio.h>

int main(void)
{
    int edad;
    double altura;
    char inicial;

    printf("Introduce tu edad: ");
    scanf("%d", &edad);
    printf("Introduce tu altura en metros: ");
    scanf("%lf", &altura);
    printf("Introduce la inicial de tu nombre: ");
    scanf(" %c", &inicial);

    printf("\nTienes %d anios, mides %.2f m y tu inicial es %c.\n",
           edad, altura, inicial);
    return 0;
}
```

Leer/escribir un solo carácter: `getchar()` lee un carácter y `putchar(c)` lo escribe.

Leer un texto (cadena): `char nombre[30]; scanf("%s", nombre);` (sin `&`, y solo lee **hasta el primer espacio**). Más en el Capítulo 11.

Errores típicos: olvidar el `&` (el programa se cae); usar `%f` para leer un `double`; no poner el espacio en `" %c"`; usar `%d` para imprimir un `double` (sale basura).

✿ Capítulo 6 · Estructuras secuenciales

Una **estructura secuencial** es un programa que va de arriba abajo, sin decisiones ni repeticiones. El esquema es siempre el mismo:

1) Declarar variables → 2) Pedir datos → 3) Calcular → 4) Mostrar resultados.

Ejemplo 6.1 · Área y perímetro de un círculo.

```
#include <stdio.h>
#define PI 3.14159265

int main(void)
{
    double radio, area, perimetro;

    printf("Radio: ");
    scanf("%lf", &radio);

    area = PI * radio * radio;
    perimetro = 2 * PI * radio;

    printf("Area = %.2f\n", area);
    printf("Perimetro = %.2f\n", perimetro);
    return 0;
}
```

Ejemplo 6.2 · De Celsius a Fahrenheit. (F = C · 9/5 + 32.) Ojo: usa `9.0/5.0` y no `9/5` (que da 1 por ser división entera).

```
#include <stdio.h>

int main(void)
{
    double c, f;
    printf("Grados Celsius: ");
    scanf("%lf", &c);
    f = c * 9.0 / 5.0 + 32.0;
    printf("%.1f C = %.1f F\n", c, f);
    return 0;
}
```

Ejemplo 6.3 · Intercambiar dos variables (se necesita una variable auxiliar, como cuando cambias el contenido de dos vasos usando un tercero).

```c
#include <stdio.h>

int main(void)
{
    int a = 3, b = 8, aux;
    printf("Antes:   a=%d b=%d\n", a, b);
    aux = a;
    a = b;
    b = aux;
    printf("Despues: a=%d b=%d\n", a, b);
    return 0;
}
```

Ejemplo 6.4 · Convertir segundos a horas, minutos y segundos (con `/` y `%`).

```c
#include <stdio.h>

int main(void)
{
    int total, h, m, s;
    printf("Segundos totales: ");
    scanf("%d", &total);
    h = total / 3600;
    m = (total % 3600) / 60;
    s = total % 60;
    printf("%d s = %d h %d min %d s\n", total, h, m, s);
    return 0;
}
```

Ejemplo 6.5 · Raíz cuadrada y potencias con `math.h` (¡compila con `-lm`!).

```c
#include <stdio.h>
#include <math.h>

int main(void)
{
    double x;
    printf("Numero: ");
    scanf("%lf", &x);
    printf("sqrt = %.3f\n", sqrt(x));
    printf("x al cubo = %.3f\n", pow(x, 3));
    printf("valor absoluto = %.3f\n", fabs(x));
    return 0;
}
```

Funciones útiles de `math.h`: `sqrt(x)` raíz, `pow(x, y)` x elevado a y, `fabs(x)` valor absoluto de decimales (`abs(n)` para enteros, en `stdlib.h`), `sin cos tan`, `exp`, `log` (neperiano), `log10`, `floor` (redondea hacia abajo), `ceil` (hacia arriba), `round`.

✿ Capítulo 7 · Estructuras selectivas (if-else y switch)

Hasta ahora el programa hacía siempre lo mismo. Con las **selectivas** el programa **decide** según una condición.

`if` simple:

```
if (condición) {
    // se ejecuta solo si la condición es verdadera
}
```

`if-else`:

```
if (nota >= 5) {
    printf("Aprobado\n");
} else {
    printf("Suspenso\n");
}
```

`else if` (varias opciones, se prueba de arriba abajo y se ejecuta la primera que cumple):

```
if (nota >= 9)      printf("Sobresaliente\n");
else if (nota >= 7) printf("Notable\n");
else if (nota >= 5) printf("Aprobado\n");
else                printf("Suspenso\n");
```

Reglas:

- La condición va **entre paréntesis** y **sin** `;` después. `if (x > 0);` es un error muy típico: ese `;` hace que el `if` no haga nada.
- Si el bloque tiene **una sola instrucción**, las llaves son opcionales, pero **ponlas siempre**: evitan errores.
- `=` asigna, `==` compara (ya lo vimos).
- Se pueden **anidar** (un if dentro de otro).

Ejemplo 7.1 · Par o impar.

```c
#include <stdio.h>

int main(void)
{
    int n;
    printf("Numero: ");
    scanf("%d", &n);
    if (n % 2 == 0) {
        printf("%d es par\n", n);
    } else {
        printf("%d es impar\n", n);
    }
    return 0;
}
```

Ejemplo 7.2 · El mayor de tres números.

```c
#include <stdio.h>

int main(void)
{
    int a, b, c, mayor;
    printf("Tres numeros: ");
    scanf("%d %d %d", &a, &b, &c);
    mayor = a;
    if (b > mayor) mayor = b;
    if (c > mayor) mayor = c;
    printf("El mayor es %d\n", mayor);
    return 0;
}
```

Ejemplo 7.3 · Ecuación de segundo grado (anidamos según el discriminante).

```c
#include <stdio.h>
#include <math.h>

int main(void)
{
    double a, b, c, d;
    printf("Coeficientes a b c: ");
    scanf("%lf %lf %lf", &a, &b, &c);
    if (a == 0) {
        printf("No es de segundo grado\n");
    } else {
        d = b * b - 4 * a * c;
        if (d > 0) {
            printf("x1 = %.3f\n", (-b + sqrt(d)) / (2 * a));
            printf("x2 = %.3f\n", (-b - sqrt(d)) / (2 * a));
        } else if (d == 0) {
            printf("Solucion doble: x = %.3f\n", -b / (2 * a));
        } else {
            printf("No hay soluciones reales\n");
        }
    }
    return 0;
}
```

Ejemplo 7.4 · Año bisiesto. Es bisiesto si es divisible por 4 **y no** por 100, **o bien** es divisible por 400.

```c
#include <stdio.h>

int main(void)
{
    int anio;
    printf("Anio: ");
    scanf("%d", &anio);
    if ((anio % 4 == 0 && anio % 100 != 0) || anio % 400 == 0) {
        printf("%d es bisiesto\n", anio);
    } else {
        printf("%d no es bisiesto\n", anio);
    }
    return 0;
}
```

`switch` : elegir entre muchos casos concretos. Sirve cuando comparas **una variable entera o carácter** con **valores exactos**.

```c
switch (variable) {
    case valor1:
        // instrucciones
        break;
    case valor2:
        // instrucciones
        break;
    default:
        // si no coincide ninguno
}
```

- **`break` es imprescindible**: sin él, el programa "se cae" al siguiente `case` y ejecuta también sus instrucciones.
- `default` es opcional (el "si no, entonces...").
- No vale con decimales ni con rangos (`>= 5`): para eso, `if` .
- Se pueden juntar casos: `case 'a': case 'e': ...` comparten las instrucciones.

Ejemplo 7.5 · Calculadora con switch.

```c
#include <stdio.h>

int main(void)
{
    double a, b;
    char op;
    printf("Operacion (ej: 3 + 4): ");
    scanf("%lf %c %lf", &a, &op, &b);
    switch (op) {
        case '+': printf("= %.2f\n", a + b); break;
        case '-': printf("= %.2f\n", a - b); break;
        case '*': printf("= %.2f\n", a * b); break;
        case '/':
            if (b != 0) printf("= %.2f\n", a / b);
            else        printf("No se puede dividir entre 0\n");
            break;
        default:
            printf("Operador desconocido\n");
    }
    return 0;
}
```

Ejemplo 7.6 · Vocal o consonante (casos agrupados).

```c
#include <stdio.h>

int main(void)
{
    char c;
    printf("Letra minuscula: ");
    scanf(" %c", &c);
    switch (c) {
        case 'a': case 'e': case 'i': case 'o': case 'u':
            printf("Es una vocal\n");
            break;
        default:
            printf("Es una consonante (o no es letra)\n");
    }
    return 0;
}
```

✿ Capítulo 8 · Estructuras iterativas (bucles)

Un **bucle** repite instrucciones. Hay tres tipos.

`while` : repite *mientras* la condición sea verdadera (puede no ejecutarse ni una vez):

```
while (condición) {
    // cuerpo
}
```

`do-while` : igual, pero comprueba al final → el cuerpo se ejecuta **al menos una vez** (ideal para menús y validar entradas):

```
do {
    // cuerpo
} while (condición);     // ← aquí SÍ lleva ;
```

`for` : cuando sabes cuántas veces (tiene contador):

```
for (inicio; condición; paso) {
    // cuerpo
}
// Ejemplo: i = 1, 2, 3, ... , 10
for (int i = 1; i <= 10; i++) {
    printf("%d ", i);
}
```

El `for` equivale a: `inicio; while (condición) { cuerpo; paso; }`.

Dos patrones que usarás constantemente:

- ✿ **Contador:** cuenta cuántas veces pasa algo → `contador++;` (empieza en 0).
- ✿ **Acumulador:** va sumando valores → `suma += x;` (empieza en 0; para productos, empieza en **1**).

Control del bucle: `break;` sale del bucle inmediatamente; `continue;` salta al siguiente paso del bucle.

⚠ **Bucle infinito:** si la condición nunca se vuelve falsa, el programa no acaba (se corta con **Ctrl+C**). Casi siempre es que olvidaste actualizar la variable de la condición.

Ejemplo 8.1 · Suma de los números del 1 al n.

```c
#include <stdio.h>

int main(void)
{
    int n, suma = 0;
    printf("n: ");
    scanf("%d", &n);
    for (int i = 1; i <= n; i++) {
        suma += i;
    }
    printf("1 + 2 + ... + %d = %d\n", n, suma);
    return 0;
}
```

Ejemplo 8.2 · Tabla de multiplicar.

```c
#include <stdio.h>

int main(void)
{
    int n;
    printf("Tabla del: ");
    scanf("%d", &n);
    for (int i = 1; i <= 10; i++) {
        printf("%d x %2d = %3d\n", n, i, n * i);
    }
    return 0;
}
```

Ejemplo 8.3 · Factorial (n! = 1·2·…·n; acumulador que empieza en 1).

```c
#include <stdio.h>

int main(void)
{
    int n;
    long long f = 1;
    printf("n: ");
    scanf("%d", &n);
    for (int i = 2; i <= n; i++) {
        f *= i;
    }
    printf("%d! = %lld\n", n, f);
    return 0;
}
```

Ejemplo 8.4 · Validar una entrada con do-while (repite hasta que el dato sea correcto).

```c
#include <stdio.h>

int main(void)
{
    int nota;
    do {
        printf("Nota (0-10): ");
        scanf("%d", &nota);
        if (nota < 0 || nota > 10) {
            printf("Valor no valido, repite.\n");
        }
    } while (nota < 0 || nota > 10);
    printf("Has introducido %d\n", nota);
    return 0;
}
```

Ejemplo 8.5 · Media con centinela (se leen números hasta que el usuario escribe 0, que es la "señal de parada").

```c
#include <stdio.h>

int main(void)
{
    double x, suma = 0;
    int cuantos = 0;
    printf("Numeros (0 para terminar): ");
    scanf("%lf", &x);
    while (x != 0) {
        suma += x;
        cuantos++;
        scanf("%lf", &x);
    }
    if (cuantos > 0) {
        printf("Media = %.2f\n", suma / cuantos);
    } else {
        printf("No has escrito ningun numero\n");
    }
    return 0;
}
```

Ejemplo 8.6 · ¿Es primo? (probamos los divisores; con `break` salimos al encontrar uno).

```c
#include <stdio.h>

int main(void)
{
    int n, primo = 1;
    printf("Numero: ");
    scanf("%d", &n);
    if (n < 2) primo = 0;
    for (int d = 2; d * d <= n; d++) {
        if (n % d == 0) {
            primo = 0;
            break;
        }
    }
    printf("%d %s primo\n", n, primo ? "es" : "no es");
    return 0;
}
```

(`d * d <= n` basta porque si un número tiene divisor, tiene uno ≤ su raíz.)

Ejemplo 8.7 · Bucles anidados: dibujar un triángulo. El bucle de fuera son las filas, el de dentro las columnas.

```
#include <stdio.h>

int main(void)
{
    int n;
    printf("Alto: ");
    scanf("%d", &n);
    for (int fila = 1; fila <= n; fila++) {
        for (int col = 1; col <= fila; col++) {
            printf("*");
        }
        printf("\n");
    }
    return 0;
}
```

Para n = 4 sale:

```
*
**
***
****
```

Ejemplo 8.8 · Menú con do-while y switch (la estructura típica de las prácticas).

```
#include <stdio.h>

int main(void)
{
    int opcion;
    do {
        printf("\n--- MENU ---\n");
        printf("1. Saludar\n");
        printf("2. Despedirse\n");
        printf("0. Salir\n");
        printf("Opcion: ");
        scanf("%d", &opcion);
        switch (opcion) {
            case 1: printf("Hola!\n"); break;
            case 2: printf("Adios!\n"); break;
            case 0: printf("Fin del programa\n"); break;
            default: printf("Opcion no valida\n");
        }
    } while (opcion != 0);
    return 0;
}
```

¿Cuál bucle elijo? Sé cuántas veces → `for`. Repetir mientras pase algo, quizá 0 veces → `while`. Quiero que se haga al menos una vez (menú, validar) → `do-while`.

✿ Capítulo 9 · Variables locales y globales

El **ámbito** (*scope*) de una variable es la zona del programa donde existe y se puede usar.

- **Variable local:** declarada **dentro** de una función (o de un bloque `{ }`). Solo existe ahí; al terminar la función, **desaparece**.
- **Variable global:** declarada **fuera** de todas las funciones (arriba del todo). Se puede usar en cualquier función.

```
#include <stdio.h>

int total = 0;            // GLOBAL: la ven todas las funciones

void sumar(int x)
{
    int doble = 2 * x;    // LOCAL: solo existe dentro de sumar
    total += doble;
}

int main(void)
{
    sumar(3);
    sumar(4);
    printf("total = %d\n", total);   // 14
    // printf("%d", doble);          // ERROR: doble no existe aquí
    return 0;
}
```

Reglas importantes:

- Una variable local **no es visible** desde otra función. Dos funciones pueden tener variables locales con el **mismo nombre** y no se molestan entre sí (son cajitas distintas).
- Si una local tiene el mismo nombre que una global, **dentro de esa función manda la local** (*sombrea* a la global).
- Las variables de un `for` (`int i` dentro del paréntesis) existen **solo** en ese `for`.
- Las locales **no se inicializan solas** (basura); las globales **sí** (valen 0).

Ejemplo 9.1 · Local contra global del mismo nombre.

```c
#include <stdio.h>

int x = 10;                  // global

void f(void)
{
    int x = 99;              // local: tapa a la global dentro de f
    printf("En f: x = %d\n", x);
}

int main(void)
{
    f();
    printf("En main: x = %d\n", x);   // la global: 10
    return 0;
}
```

Salida: `En f: x = 99` y `En main: x = 10`.

> **Consejo de estilo: usa locales siempre que puedas** y pasa los datos a las funciones como parámetros. Las globales hacen los programas difíciles de entender y de depurar (cualquiera puede cambiarlas). Se reservan para constantes o casos muy justificados.

✿ Capítulo 10 · Funciones

Una **función** es un trozo de código con nombre que haces **una vez** y puedes **usar muchas veces**. Divide el programa en partes pequeñas y manejables. `main` es una función más; ya sabes usar otras (`printf`, `sqrt`).

Anatomía:

```
tipo_que_devuelve nombre(tipo parametro1, tipo parametro2)
{
    // instrucciones
    return valor;
}
```

- ✿ **Parámetros:** los datos que le pasas a la función para que trabaje.
- ✿ **`return`:** devuelve un resultado y termina la función.
- ✿ **`void`:** si no devuelve nada (solo hace algo, como escribir) o no recibe nada.

Ejemplo 10.1 · Una función que devuelve un valor.

```
#include <stdio.h>

int cuadrado(int n)
{
    return n * n;
}

int main(void)
{
    int r = cuadrado(7);
    printf("7 al cuadrado = %d\n", r);
    printf("5 al cuadrado = %d\n", cuadrado(5));
    return 0;
}
```

Prototipos. C lee el archivo de arriba abajo: una función tiene que estar **declarada antes de usarla**. Dos opciones: poner las funciones antes de `main`, o (lo habitual) poner arriba solo el **prototipo** (la cabecera acabada en `;`) y las definiciones después de `main`:

```
#include <stdio.h>

int mayor(int a, int b);          // prototipo
void saludar(void);               // prototipo

int main(void)
{
    saludar();
    printf("El mayor de 4 y 9 es %d\n", mayor(4, 9));
    return 0;
}

int mayor(int a, int b)           // definición
{
    if (a > b) return a;
    return b;
}

void saludar(void)
{
    printf("Hola desde una funcion!\n");
}
```

Paso por valor (¡clave!). Cuando llamas a una función, ella recibe una **copia** de los valores. Si modifica el parámetro, **la variable original no cambia**:

```
#include <stdio.h>

void intenta_cambiar(int x)
{
    x = 100;                      // solo cambia la copia
}

int main(void)
{
    int a = 5;
    intenta_cambiar(a);
    printf("a = %d\n", a);        // sigue valiendo 5
    return 0;
}
```

Para que una función **modifique** variables de quien la llama hacen falta **punteros** (Capítulo 13; es lo que hace `scanf` con el `&`). Por ahora, si necesitas "sacar" un resultado, usa `return`. Los **vectores** sí se modifican dentro de funciones (Capítulo 11).

Ejemplo 10.2 · Funciones útiles: factorial, es_primo y potencia.

```c
#include <stdio.h>

long long factorial(int n)
{
    long long f = 1;
    for (int i = 2; i <= n; i++) f *= i;
    return f;
}

int es_primo(int n)
{
    if (n < 2) return 0;
    for (int d = 2; d * d <= n; d++) {
        if (n % d == 0) return 0;
    }
    return 1;
}

double potencia(double base, int exp)
{
    double r = 1;
    for (int i = 0; i < exp; i++) r *= base;
    return r;
}

int main(void)
{
    printf("5! = %lld\n", factorial(5));
    printf("¿13 es primo? %d\n", es_primo(13));
    printf("2^10 = %.0f\n", potencia(2, 10));
    return 0;
}
```

Ejemplo 10.3 · Funciones que se llaman entre sí. Una función puede usar otras: aquí el área del círculo usa `cuadrado`.

```c
#include <stdio.h>

double cuadrado(double x)
{
    return x * x;
}

double area_circulo(double r)
{
    return 3.14159265 * cuadrado(r);
}

int main(void)
{
    printf("Area con r=2: %.2f\n", area_circulo(2));
    return 0;
}
```

Ejemplo 10.4 · Recursividad (una función que se llama a sí misma). Siempre necesita un **caso base** que pare la recursión.

```c
#include <stdio.h>

long long fact(int n)
{
    if (n <= 1) return 1;        // caso base
    return n * fact(n - 1);      // caso recursivo
}

int main(void)
{
    printf("6! = %lld\n", fact(6));
    return 0;
}
```

> **Por qué usar funciones:** evitan repetir código, hacen el programa legible (`es_primo(n)` se entiende de un vistazo) y permiten probar cada parte por separado.

Errores típicos: olvidar el `return` en una función que devuelve algo; llamar con el número de argumentos incorrecto; poner `;` tras la cabecera de la definición; usar la función antes de declararla; creer que modificar un parámetro cambia la variable original.

✿ Capítulo 11 · Vectores y matrices

Un **vector** (array) es una fila de **muchas variables del mismo tipo** con un solo nombre. Si tienes 30 notas, no creas `nota1, nota2, …, nota30`: creas un vector de 30 posiciones.

Declarar e inicializar:

```
int notas[5];                          // 5 enteros (con basura dentro)
int v[5] = {4, 8, 15, 16, 23};         // inicializado
int w[] = {1, 2, 3};                   // tamaño 3 deducido
int ceros[100] = {0};                  // 100 posiciones, todas a 0
```

Índices desde 0. Un vector de 5 elementos tiene posiciones **0, 1, 2, 3, 4**. **No existe** `v[5]`: salirte del vector no da error al compilar, pero hace cosas raras (o rompe el programa). Es el fallo más peligroso de C.

```
v[0] = 10;                // primer elemento
printf("%d", v[2]);       // tercer elemento
v[4] = v[0] + v[1];       // último elemento
```

Recorrer con `for` (la forma estándar: `i` va de 0 a **menor que** el tamaño):

```
for (int i = 0; i < N; i++) {
    // usar v[i]
}
```

> **Truco:** define el tamaño con una constante (`#define N 5`) y así lo cambias en un solo sitio.

Ejemplo 11.1 · Leer, mostrar y calcular la media.

```
#include <stdio.h>
#define N 5

int main(void)
{
    double nota[N], suma = 0;

    for (int i = 0; i < N; i++) {
        printf("Nota %d: ", i + 1);
        scanf("%lf", &nota[i]);
        suma += nota[i];
    }

    printf("\nNotas: ");
    for (int i = 0; i < N; i++) {
        printf("%.1f ", nota[i]);
    }
    printf("\nMedia = %.2f\n", suma / N);
    return 0;
}
```

(Fíjate en el `&nota[i]`: cada posición es una variable y se lee con `&`.)

Ejemplo 11.2 · Máximo, mínimo y posición del máximo. Truco: se parte del primer elemento y se compara con los demás.

```
#include <stdio.h>
#define N 6

int main(void)
{
    int v[N] = {12, 7, 45, 3, 29, 45};
    int max = v[0], min = v[0], pos = 0;

    for (int i = 1; i < N; i++) {
        if (v[i] > max) { max = v[i]; pos = i; }
        if (v[i] < min) { min = v[i]; }
    }
    printf("Maximo = %d (posicion %d)\n", max, pos);
    printf("Minimo = %d\n", min);
    return 0;
}
```

Ejemplo 11.3 · Búsqueda: ¿está un número en el vector?

```
#include <stdio.h>
#define N 6

int main(void)
{
    int v[N] = {12, 7, 45, 3, 29, 8};
    int buscado, pos = -1;

    printf("Que numero buscas? ");
    scanf("%d", &buscado);
    for (int i = 0; i < N; i++) {
        if (v[i] == buscado) {
            pos = i;
            break;
        }
    }
    if (pos >= 0) printf("Esta en la posicion %d\n", pos);
    else          printf("No esta\n");
    return 0;
}
```

Ejemplo 11.4 · Invertir un vector (intercambiando extremos).

```c
#include <stdio.h>
#define N 6

int main(void)
{
    int v[N] = {1, 2, 3, 4, 5, 6}, aux;
    for (int i = 0; i < N / 2; i++) {
        aux = v[i];
        v[i] = v[N - 1 - i];
        v[N - 1 - i] = aux;
    }
    for (int i = 0; i < N; i++) printf("%d ", v[i]);
    printf("\n");
    return 0;
}
```

Ejemplo 11.5 · Ordenar con el método de la burbuja. Se recorre el vector comparando vecinos; si están al revés, se intercambian. Tras cada pasada, el mayor "sube" al final.

```c
#include <stdio.h>
#define N 6

int main(void)
{
    int v[N] = {29, 7, 45, 3, 12, 8}, aux;

    for (int pasada = 0; pasada < N - 1; pasada++) {
        for (int i = 0; i < N - 1 - pasada; i++) {
            if (v[i] > v[i + 1]) {
                aux = v[i];
                v[i] = v[i + 1];
                v[i + 1] = aux;
            }
        }
    }
    for (int i = 0; i < N; i++) printf("%d ", v[i]);
    printf("\n");
    return 0;
}
```

Vectores y funciones. A una función se le pasa el vector y **su tamaño** (la función no sabe cuánto mide). **Los vectores sí se modifican dentro de la función** (no se copian, se pasa el original).

```c
#include <stdio.h>

double media(double v[], int n)
{
    double suma = 0;
    for (int i = 0; i < n; i++) suma += v[i];
    return suma / n;
}

void duplicar(int v[], int n)          // modifica el vector original
{
    for (int i = 0; i < n; i++) v[i] *= 2;
}

int main(void)
{
    double notas[4] = {5, 7.5, 9, 6};
    int w[3] = {1, 2, 3};

    printf("Media = %.2f\n", media(notas, 4));
    duplicar(w, 3);
    printf("%d %d %d\n", w[0], w[1], w[2]);     // 2 4 6
    return 0;
}
```

Matrices (vectores de dos dimensiones). Una tabla con **filas** y **columnas**. Se accede con **dos índices**: `m[fila][columna]`.

```c
int m[2][3] = { {1, 2, 3},
                {4, 5, 6} };        // 2 filas, 3 columnas
m[0][2] = 9;                        // fila 0, columna 2
```

Se recorren con **dos `for` anidados**: el de fuera las filas, el de dentro las columnas.

Ejemplo 11.6 · Leer una matriz, mostrarla y sumar sus elementos.

```c
#include <stdio.h>
#define F 2
#define C 3

int main(void)
{
    int m[F][C], suma = 0;

    for (int i = 0; i < F; i++) {
        for (int j = 0; j < C; j++) {
            printf("m[%d][%d] = ", i, j);
            scanf("%d", &m[i][j]);
            suma += m[i][j];
        }
    }
    printf("\nLa matriz:\n");
    for (int i = 0; i < F; i++) {
        for (int j = 0; j < C; j++) {
            printf("%4d", m[i][j]);
        }
        printf("\n");
    }
    printf("Suma de todos = %d\n", suma);
    return 0;
}
```

Ejemplo 11.7 · Suma de dos matrices y traspuesta. (Sumar: elemento a elemento. Trasponer: filas ↔ columnas.)

```c
#include <stdio.h>
#define N 3

int main(void)
{
    int a[N][N] = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
    int b[N][N] = {{9, 8, 7}, {6, 5, 4}, {3, 2, 1}};
    int s[N][N], t[N][N];

    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            s[i][j] = a[i][j] + b[i][j];
            t[j][i] = a[i][j];
        }
    }
    printf("Suma:\n");
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) printf("%4d", s[i][j]);
        printf("\n");
    }
    printf("Traspuesta de a:\n");
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) printf("%4d", t[i][j]);
        printf("\n");
    }
    return 0;
}
```

Matrices y funciones: hay que indicar el **número de columnas** en el parámetro (`int m[][C]`) y pasar las filas aparte.

```
#include <stdio.h>
#define C 3

int suma_matriz(int m[][C], int filas)
{
    int s = 0;
    for (int i = 0; i < filas; i++)
        for (int j = 0; j < C; j++)
            s += m[i][j];
    return s;
}

int main(void)
{
    int m[2][C] = {{1, 2, 3}, {4, 5, 6}};
    printf("Suma = %d\n", suma_matriz(m, 2));
    return 0;
}
```

Cadenas de texto (un vector de `char`). Un texto es un vector de caracteres que **termina en `'\0'`** (carácter nulo, lo pone C solo). Por eso el vector debe tener **un hueco más** que letras.

```
#include <stdio.h>
#include <string.h>

int main(void)
{
    char nombre[30];
    printf("Tu nombre: ");
    scanf("%s", nombre);                 // sin &, lee hasta el primer espacio
    printf("Hola, %s!\n", nombre);
    printf("Tiene %zu letras\n", strlen(nombre));

    char saludo[20] = "Hola";
    strcat(saludo, " mundo");            // concatenar
    printf("%s\n", saludo);
    if (strcmp(nombre, "Ana") == 0) {    // comparar (0 = iguales)
        printf("Eres Ana\n");
    }
    return 0;
}
```

De `string.h`: `strlen(s)` longitud · `strcpy(destino, origen)` copiar · `strcat(a, b)` pegar b al final de a · `strcmp(a, b)` comparar (**nunca uses `==` con cadenas**).

Errores típicos con vectores: acceder a `v[N]` (se sale); olvidar pasar el tamaño a la función; usar `i <= N` en lugar de `i < N`; querer copiar un vector con `a = b` (no se puede: hay que copiar elemento a elemento); leer con `scanf("%d", v[i])` sin `&`.

✿ Capítulo 12 · Números aleatorios (Práctica 6)

Para simulaciones, juegos o generar datos de prueba, C trae `rand()` (en `stdlib.h`). Genera un entero "al azar" entre 0 y `RAND_MAX`.

El problema: sin más, `rand()` da **siempre la misma secuencia** cada vez que ejecutas. Hay que "sembrar" el generador **una sola vez**, al principio del programa, con la hora actual: `srand(time(NULL));` (necesita `time.h`).

Número en un rango [a, b]: `a + rand() % (b - a + 1)`.

Quiero...	Código
de 0 a 9	`rand() % 10`
de 1 a 6 (dado)	`1 + rand() % 6`
de 10 a 20	`10 + rand() % 11`
decimal entre 0 y 1	`rand() / (double)RAND_MAX`

Ejemplo 12.1 · Tirar un dado 10 veces.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main(void)
{
    srand(time(NULL));
    for (int i = 0; i < 10; i++) {
        printf("%d ", 1 + rand() % 6);
    }
    printf("\n");
    return 0;
}
```

Ejemplo 12.2 · Rellenar un vector con aleatorios y buscar el máximo.

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define N 10

int main(void)
{
    int v[N], max;
    srand(time(NULL));
    for (int i = 0; i < N; i++) {
        v[i] = rand() % 100;               // 0..99
        printf("%d ", v[i]);
    }
    max = v[0];
    for (int i = 1; i < N; i++)
        if (v[i] > max) max = v[i];
    printf("\nMaximo = %d\n", max);
    return 0;
}
```

Ejemplo 12.3 · Adivina el número (junta bucles, if y aleatorios).

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main(void)
{
    int secreto, intento, intentos = 0;
    srand(time(NULL));
    secreto = 1 + rand() % 100;
    printf("He pensado un numero del 1 al 100.\n");
    do {
        printf("Tu intento: ");
        scanf("%d", &intento);
        intentos++;
        if (intento < secreto)      printf("Mas alto\n");
        else if (intento > secreto) printf("Mas bajo\n");
    } while (intento != secreto);
    printf("Acertaste en %d intentos!\n", intentos);
    return 0;
}
```

Ejemplo 12.4 · Matriz aleatoria con una función.

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define F 3
#define C 4

void rellenar(int m[][C], int filas)
{
    for (int i = 0; i < filas; i++)
        for (int j = 0; j < C; j++)
            m[i][j] = rand() % 10;
}

void mostrar(int m[][C], int filas)
{
    for (int i = 0; i < filas; i++) {
        for (int j = 0; j < C; j++) printf("%3d", m[i][j]);
        printf("\n");
    }
}

int main(void)
{
    int m[F][C];
    srand(time(NULL));
    rellenar(m, F);
    mostrar(m, F);
    return 0;
}
```

Importante: `srand` se llama **una vez**, no dentro del bucle (si no, saldría siempre el mismo número).

✿ Capítulo 13 · Punteros

Un **puntero** es una variable que guarda la **dirección de memoria** de otra variable (en vez de guardar un dato directamente).

Analogía: la memoria es una calle con casas numeradas. `int a = 5;` es una casa donde vive el 5. Un puntero es un papelito con **el número de la casa**.

Los dos símbolos clave:

- ✿ `&a` → "la dirección de `a`".
- ✿ `*p` → "lo que hay en la dirección que guarda `p`" (se llama **desreferenciar**).

Declarar un puntero: `tipo *nombre;` → `int *p;` es "p es un puntero a int".

Ejemplo 13.1 · Primer contacto.

```
#include <stdio.h>

int main(void)
{
    int a = 5;
    int *p = &a;                // p apunta a a

    printf("a vale %d\n", a);
    printf("*p vale %d\n", *p);  // 5 (lo mismo que a)

    *p = 20;                    // cambiar a través del puntero
    printf("Ahora a vale %d\n", a);   // 20

    printf("La direccion de a es %p\n", (void *)&a);
    return 0;
}
```

NULL: un puntero que no apunta a nada se inicializa a `NULL`. **Nunca** uses `*p` si `p` es `NULL` o no está inicializado: el programa se cae (*segmentation fault*).

```
int *p = NULL;
if (p != NULL) { printf("%d", *p); }
```

¿Para qué sirven?

1) Funciones que modifican variables de quien las llama (paso por *referencia*). Recuerda: las funciones reciben copias. Si les pasas **la dirección**, pueden cambiar el original.

Ejemplo 13.2 · Intercambiar dos variables con una función.

```c
#include <stdio.h>

void intercambiar(int *x, int *y)
{
    int aux = *x;
    *x = *y;
    *y = aux;
}

int main(void)
{
    int a = 3, b = 8;
    intercambiar(&a, &b);
    printf("a=%d b=%d\n", a, b);    // a=8 b=3
    return 0;
}
```

Por eso `scanf("%d", &edad)` lleva `&`: le pasas la dirección de `edad` para que pueda escribir dentro.

2) Devolver varios resultados (un `return` solo da uno).

Ejemplo 13.3 · Cociente y resto a la vez.

```c
#include <stdio.h>

void dividir(int a, int b, int *cociente, int *resto)
{
    *cociente = a / b;
    *resto = a % b;
}

int main(void)
{
    int c, r;
    dividir(17, 5, &c, &r);
    printf("17 = 5 * %d + %d\n", c, r);
    return 0;
}
```

Ejemplo 13.4 · Máximo y mínimo de un vector en una sola función.

```c
#include <stdio.h>

void max_min(int v[], int n, int *max, int *min)
{
    *max = v[0];
    *min = v[0];
    for (int i = 1; i < n; i++) {
        if (v[i] > *max) *max = v[i];
        if (v[i] < *min) *min = v[i];
    }
}

int main(void)
{
    int v[5] = {4, 9, -2, 7, 1}, mx, mn;
    max_min(v, 5, &mx, &mn);
    printf("max=%d min=%d\n", mx, mn);
    return 0;
}
```

Punteros y vectores. El nombre de un vector **es** la dirección de su primer elemento. Por eso una función puede modificar un vector sin que le pases `&`: ya le estás pasando una dirección.

```
int v[3] = {10, 20, 30};
int *p = v;          // igual que p = &v[0]
// p[0], p[1], p[2]  ==  v[0], v[1], v[2]
// *(p + 1)          ==  v[1]   (aritmética de punteros)
```

Aritmética de punteros: `p + 1` no suma 1 byte, sino **el tamaño de un elemento** (avanza al siguiente elemento del vector).

Ejemplo 13.5 · Recorrer un vector con un puntero.

```
#include <stdio.h>

int main(void)
{
    int v[5] = {1, 2, 3, 4, 5};
    int *p;
    for (p = v; p < v + 5; p++) {
        printf("%d ", *p);
    }
    printf("\n");
    return 0;
}
```

Errores típicos con punteros: usar un puntero sin inicializar; olvidar el `*` o el `&` (`p = 5` en vez de `*p = 5`); devolver la dirección de una variable local (desaparece al acabar la función); confundir `int *p` (declaración) con `*p` (uso).

✿ Capítulo 14 · Cadenas de texto a fondo

Una **cadena** es un vector de `char` que termina en el carácter nulo `'\0'`. Por eso `char s[5]` guarda como máximo **4 letras**.

```
char a[10] = "Hola";            // H o l a \0 y sobra espacio
char b[] = "Mundo";             // tamaño 6 (5 letras + '\0')
```

Leer cadenas.

- ✿ `scanf("%s", cad);` → sin `&`, lee **hasta el primer espacio**.
- ✿ `fgets(cad, tamaño, stdin);` → lee **una línea entera** (con espacios) y **conserva el `\n`** del final. Es la forma segura.

```
char frase[100];
fgets(frase, sizeof(frase), stdin);
frase[strcspn(frase, "\n")] = '\0';     // quita el \n final
```

Funciones de `<string.h>`:

Función	Qué hace
`strlen(s)`	longitud (sin contar `'\0'`)
`strcpy(dest, orig)`	copia `orig` en `dest` (¡`dest` debe tener sitio!)
`strncpy(dest, orig, n)`	copia como mucho `n` caracteres
`strcat(a, b)`	pega `b` al final de `a`
`strcmp(a, b)`	compara: 0 si iguales, <0 si `a` va antes, >0 si después
`strchr(s, c)`	busca un carácter (devuelve puntero o `NULL`)
`strstr(s, sub)`	busca una subcadena

Funciones de `<ctype.h>` (sobre un carácter): `isalpha(c)` letra · `isdigit(c)` dígito · `isspace(c)` espacio · `isupper(c)` / `islower(c)` · `toupper(c)` / `tolower(c)` convierten.

⚠ **No se puede** hacer `a = "hola";` ni `if (a == b)` con cadenas. Hay que usar `strcpy` y `strcmp`.

Ejemplo 14.1 · Contar vocales y mayúsculas en una frase.

```c
#include <stdio.h>
#include <string.h>
#include <ctype.h>

int main(void)
{
    char frase[100];
    int vocales = 0, mayusculas = 0;

    printf("Escribe una frase: ");
    fgets(frase, sizeof(frase), stdin);

    for (int i = 0; frase[i] != '\0'; i++) {
        char c = tolower(frase[i]);
        if (c == 'a' || c == 'e' || c == 'i' || c == 'o' || c == 'u')
            vocales++;
        if (isupper(frase[i]))
            mayusculas++;
    }
    printf("Vocales: %d, mayusculas: %d\n", vocales, mayusculas);
    return 0;
}
```

Ejemplo 14.2 · ¿Es un palíndromo? (se lee igual al derecho y al revés)

```c
#include <stdio.h>
#include <string.h>

int main(void)
{
    char s[50];
    int n, es = 1;

    printf("Palabra: ");
    scanf("%49s", s);
    n = strlen(s);
    for (int i = 0; i < n / 2; i++) {
        if (s[i] != s[n - 1 - i]) {
            es = 0;
            break;
        }
    }
    printf("%s %s un palindromo\n", s, es ? "es" : "no es");
    return 0;
}
```

Ejemplo 14.3 · Pasar a mayúsculas con una función (las cadenas, como los vectores, se modifican dentro de la función).

```c
#include <stdio.h>
#include <ctype.h>

void a_mayusculas(char s[])
{
    for (int i = 0; s[i] != '\0'; i++)
        s[i] = toupper(s[i]);
}

int main(void)
{
    char texto[] = "hola mundo";
    a_mayusculas(texto);
    printf("%s\n", texto);        // HOLA MUNDO
    return 0;
}
```

Ejemplo 14.4 · Contar palabras. Una palabra empieza cuando hay una letra tras un espacio.

```c
#include <stdio.h>
#include <ctype.h>

int main(void)
{
    char s[] = "esto es una frase de prueba";
    int palabras = 0, dentro = 0;
    for (int i = 0; s[i] != '\0'; i++) {
        if (!isspace(s[i]) && !dentro) {
            palabras++;
            dentro = 1;
        } else if (isspace(s[i])) {
            dentro = 0;
        }
    }
    printf("%d palabras\n", palabras);
    return 0;
}
```

Convertir texto ↔ número (`<stdlib.h>`): `atoi("123")` → 123, `atof("3.5")` → 3.5. Al revés, `sprintf(cad, "%d", n)` escribe el número en una cadena.

✿ Capítulo 15 · Memoria dinámica

Hasta ahora el tamaño de un vector se fija **al escribir el programa** (`int v[100]`). Con **memoria dinámica** decides el tamaño **mientras el programa corre** (por ejemplo, lo que escriba el usuario).

Las funciones (`<stdlib.h>`):

- ✿ `malloc(bytes)` → reserva memoria y devuelve un puntero (o `NULL` si no hay).
- ✿ `calloc(n, tamaño)` → reserva `n` elementos **y los pone a 0**.
- ✿ `realloc(p, nuevo_tamaño)` → cambia el tamaño de lo reservado.
- ✿ `free(p)` → **libera** la memoria cuando ya no la necesitas.

Regla de oro: todo `malloc` o `calloc` debe tener su `free`. Si no, hay una **fuga de memoria**.

Ejemplo 15.1 · Vector de tamaño elegido por el usuario.

```
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int n;
    printf("Cuantos numeros? ");
    scanf("%d", &n);

    int *v = malloc(n * sizeof(int));
    if (v == NULL) {
        printf("No hay memoria\n");
        return 1;
    }

    for (int i = 0; i < n; i++) {
        printf("v[%d] = ", i);
        scanf("%d", &v[i]);
    }
    int suma = 0;
    for (int i = 0; i < n; i++) suma += v[i];
    printf("Suma = %d\n", suma);

    free(v);                      // liberar
    return 0;
}
```

Ejemplo 15.2 · Ampliar un vector con `realloc` .

```c
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int *v = calloc(3, sizeof(int));        // 3 ceros
    if (v == NULL) return 1;
    v[0] = 1; v[1] = 2; v[2] = 3;

    int *nuevo = realloc(v, 5 * sizeof(int));
    if (nuevo == NULL) { free(v); return 1; }
    v = nuevo;
    v[3] = 4; v[4] = 5;

    for (int i = 0; i < 5; i++) printf("%d ", v[i]);
    printf("\n");
    free(v);
    return 0;
}
```

Ejemplo 15.3 · Matriz dinámica (vector de punteros a filas).

```c
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int f = 3, c = 4;
    int **m = malloc(f * sizeof(int *));
    for (int i = 0; i < f; i++)
        m[i] = malloc(c * sizeof(int));

    for (int i = 0; i < f; i++)
        for (int j = 0; j < c; j++)
            m[i][j] = i * j;

    for (int i = 0; i < f; i++) {
        for (int j = 0; j < c; j++) printf("%3d", m[i][j]);
        printf("\n");
    }

    for (int i = 0; i < f; i++) free(m[i]);   // primero las filas
    free(m);                                  // después el vector de filas
    return 0;
}
```

Errores típicos: no comprobar si `malloc` devolvió `NULL`; olvidar `free` (fuga); usar la memoria **después** de `free`; hacer `free` dos veces del mismo puntero; calcular mal el tamaño (usa siempre `n * sizeof(tipo)`).

✿ Capítulo 16 · Estructuras, typedef y enum

Un vector junta datos del **mismo tipo**. Una **estructura** (`struct`) junta datos de **tipos distintos** en un solo "paquete": por ejemplo, un alumno tiene nombre (texto), edad (entero) y nota (decimal).

```
struct Alumno {
    char nombre[30];
    int edad;
    double nota;
};
```

Se accede a cada **campo** con un punto: `a.nombre` , `a.edad` .

`typedef` pone un nombre más corto a un tipo, para no escribir `struct` cada vez:

```
typedef struct {
    char nombre[30];
    int edad;
    double nota;
} Alumno;

Alumno a = {"Lucia", 18, 8.5};      // ya se puede usar "Alumno"
```

Ejemplo 16.1 · Una estructura sencilla.

```
#include <stdio.h>
#include <string.h>

typedef struct {
    char nombre[30];
    int edad;
    double nota;
} Alumno;

int main(void)
{
    Alumno a;
    strcpy(a.nombre, "Lucia");     // las cadenas se copian con strcpy
    a.edad = 18;
    a.nota = 8.5;
    printf("%s tiene %d anios y saco un %.1f\n", a.nombre, a.edad, a.nota);
    return 0;
}
```

Vector de estructuras (la forma de guardar "una lista de alumnos"):

Ejemplo 16.2 · Notas de varios alumnos y la mejor.

```
#include <stdio.h>
#define N 3

typedef struct {
    char nombre[30];
    double nota;
} Alumno;

int main(void)
{
    Alumno clase[N];
    int mejor = 0;

    for (int i = 0; i < N; i++) {
        printf("Nombre del alumno %d: ", i + 1);
        scanf("%29s", clase[i].nombre);
        printf("Nota: ");
        scanf("%lf", &clase[i].nota);
        if (clase[i].nota > clase[mejor].nota) mejor = i;
    }
    printf("La mejor nota es de %s: %.1f\n",
           clase[mejor].nombre, clase[mejor].nota);
    return 0;
}
```

Estructuras y funciones. Si quieres **modificar** la estructura dentro de la función, pásala con **puntero** y usa la flecha `->` (es `(*p).campo` abreviado).

Ejemplo 16.3 · Función que modifica una estructura.

```
#include <stdio.h>

typedef struct {
    double x, y;
} Punto;

void mover(Punto *p, double dx, double dy)
{
    p->x += dx;
    p->y += dy;
}

void mostrar(Punto p)
{
    printf("(%.1f, %.1f)\n", p.x, p.y);
}

int main(void)
{
    Punto p = {1, 2};
    mover(&p, 3, -1);
    mostrar(p);          // (4.0, 1.0)
    return 0;
}
```

`enum`: nombres para valores constantes (empiezan en 0):

```
enum Dia { LUNES, MARTES, MIERCOLES, JUEVES, VIERNES };
enum Dia hoy = MIERCOLES;      // vale 2
```

✿ Capítulo 17 · Ficheros (leer y escribir archivos)

Para que los datos **no se pierdan** al cerrar el programa, se guardan en un **fichero**. Los pasos son siempre: **abrir → usar → cerrar.**

Abrir: `FILE *f = fopen("datos.txt", "modo");` Modos: `"r"` leer (debe existir) · `"w"` escribir (**borra** lo que hubiera) · `"a"` añadir al final. Si falla, `fopen` devuelve `NULL`: **comprueba siempre**.

Funciones principales (`<stdio.h>`), iguales que las que ya conoces pero con la `f` delante y el fichero como primer argumento:

Función	Equivale a
`fprintf(f, "...", datos)`	`printf` hacia el fichero
`fscanf(f, "...", &datos)`	`scanf` desde el fichero (devuelve cuántos leyó, o `EOF` al final)
`fgets(cad, n, f)`	leer una línea
`fclose(f)`	cerrar (¡siempre!)

Ejemplo 17.1 · Escribir un fichero.

```
#include <stdio.h>

int main(void)
{
    FILE *f = fopen("notas.txt", "w");
    if (f == NULL) {
        printf("No se pudo abrir el fichero\n");
        return 1;
    }
    fprintf(f, "Lucia 8.5\n");
    fprintf(f, "Ana 6.0\n");
    fprintf(f, "Paula 9.2\n");
    fclose(f);
    printf("Fichero escrito\n");
    return 0;
}
```

Ejemplo 17.2 · Leer el fichero anterior y calcular la media.

```c
#include <stdio.h>

int main(void)
{
    FILE *f = fopen("notas.txt", "r");
    if (f == NULL) {
        printf("No existe notas.txt (ejecuta antes el ejemplo anterior)\n");
        return 1;
    }
    char nombre[30];
    double nota, suma = 0;
    int n = 0;

    while (fscanf(f, "%29s %lf", nombre, &nota) == 2) {
        printf("%s: %.1f\n", nombre, nota);
        suma += nota;
        n++;
    }
    fclose(f);
    if (n > 0) printf("Media = %.2f\n", suma / n);
    return 0;
}
```

(`fscanf(...) == 2` significa "se han leído 2 datos"; cuando se acaba el fichero deja de cumplirse y el bucle termina.)

Ejemplo 17.3 · Añadir al final sin borrar (modo `"a"`) **y leer líneas con `fgets`.**

```c
#include <stdio.h>

int main(void)
{
    FILE *f = fopen("diario.txt", "a");
    if (f == NULL) return 1;
    fprintf(f, "Hoy he aprendido ficheros en C\n");
    fclose(f);

    f = fopen("diario.txt", "r");
    if (f == NULL) return 1;
    char linea[100];
    while (fgets(linea, sizeof(linea), f) != NULL) {
        printf("> %s", linea);
    }
    fclose(f);
    return 0;
}
```

Errores típicos: no comprobar `NULL`; abrir con `"w"` y borrar sin querer un fichero que querías conservar; olvidar `fclose` (puede que no se guarde todo); escribir la ruta mal (el fichero se busca en la carpeta desde donde ejecutas el programa).

✿ Capítulo 18 · Programas con varios archivos y preprocesador

Preprocesador: las líneas que empiezan por `#` se procesan **antes** de compilar.

- ✿ `#include <libreria.h>` → librería del sistema. `#include "mio.h"` → tu propio archivo.
- ✿ `#define NOMBRE valor` → sustituye el texto (constantes).
- ✿ **Macros con parámetros:** `#define CUADRADO(x) ((x)*(x))`. Ojo: ponle paréntesis a todo, y no uses `x++` dentro.

Dividir un programa en varios archivos. Cuando el programa crece, se separa en:

- ✿ **`.h` (cabecera):** los **prototipos** de las funciones.
- ✿ **`.c` :** las **definiciones** (el código).
- ✿ **`main.c` :** el programa principal, que hace `#include "mio.h"`.

Ejemplo 18.1 · Tres archivos.

Archivo `calculos.h` :

```
#ifndef CALCULOS_H
#define CALCULOS_H

int factorial(int n);
int es_primo(int n);

#endif
```

Archivo `calculos.c` :

```
#include "calculos.h"

int factorial(int n)
{
    int f = 1;
    for (int i = 2; i <= n; i++) f *= i;
    return f;
}

int es_primo(int n)
{
    if (n < 2) return 0;
    for (int d = 2; d * d <= n; d++)
        if (n % d == 0) return 0;
    return 1;
}
```

Archivo `main.c` :

```
#include <stdio.h>
#include "calculos.h"

int main(void)
{
    printf("5! = %d\n", factorial(5));
    printf("7 primo? %d\n", es_primo(7));
    return 0;
}
```

Compilar todo junto:

```
gcc -Wall main.c calculos.c -o programa
./programa
```

Las tres líneas `#ifndef / #define / #endif` (**guardas**) evitan que la cabecera se incluya dos veces.

`static` **dentro de una función:** una variable `static` **conserva su valor** entre llamadas.

```
#include <stdio.h>

int contador(void)
{
    static int veces = 0;      // se inicializa UNA sola vez
    veces++;
    return veces;
}

int main(void)
{
    contador(); contador();
    printf("Llamada numero %d\n", contador());   // 3
    return 0;
}
```

`const` **:** `const int MAX = 10;` no se puede modificar. En parámetros, `const char *s` promete que la función no cambiará la cadena.

Argumentos del programa: `int main(int argc, char *argv[])` recibe lo escrito al ejecutar: `./prog hola 5` → `argc` = 3, `argv[1]` = `"hola"`, `argv[2]` = `"5"` (texto: usa `atoi`).

✿ Capítulo 19 · Algoritmos clásicos

Programas que conviene saber hacer; aparecen en prácticas y exámenes.

Ejemplo 19.1 · Máximo común divisor (algoritmo de Euclides). mcd(a, b) = mcd(b, a % b) hasta que b = 0.

```
#include <stdio.h>

int mcd(int a, int b)
{
    while (b != 0) {
        int r = a % b;
        a = b;
        b = r;
    }
    return a;
}

int main(void)
{
    printf("mcd(48, 18) = %d\n", mcd(48, 18));    // 6
    return 0;
}
```

Ejemplo 19.2 · Serie de Fibonacci (cada número es la suma de los dos anteriores: 0, 1, 1, 2, 3, 5, 8…).

```
#include <stdio.h>

int main(void)
{
    int n, a = 0, b = 1;
    printf("Cuantos terminos? ");
    scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        printf("%d ", a);
        int sig = a + b;
        a = b;
        b = sig;
    }
    printf("\n");
    return 0;
}
```

Ejemplo 19.3 · De decimal a binario (dividiendo entre 2 y guardando los restos).

```c
#include <stdio.h>

int main(void)
{
    int n, bits[32], k = 0;
    printf("Numero: ");
    scanf("%d", &n);
    if (n == 0) { printf("0\n"); return 0; }
    while (n > 0) {
        bits[k++] = n % 2;
        n /= 2;
    }
    for (int i = k - 1; i >= 0; i--) printf("%d", bits[i]);
    printf("\n");
    return 0;
}
```

Ejemplo 19.4 · Búsqueda binaria (en un vector **ya ordenado**: se mira el centro y se descarta la mitad cada vez; mucho más rápido que mirar uno a uno).

```c
#include <stdio.h>

int busqueda_binaria(int v[], int n, int x)
{
    int ini = 0, fin = n - 1;
    while (ini <= fin) {
        int medio = (ini + fin) / 2;
        if (v[medio] == x) return medio;
        if (v[medio] < x) ini = medio + 1;
        else              fin = medio - 1;
    }
    return -1;
}

int main(void)
{
    int v[] = {2, 5, 8, 12, 16, 23, 38, 56};
    printf("23 esta en la posicion %d\n", busqueda_binaria(v, 8, 23));
    printf("7 esta en la posicion %d\n", busqueda_binaria(v, 8, 7));
    return 0;
}
```

Ejemplo 19.5 · Ordenación por selección (busca el mínimo del resto y lo coloca al principio).

```c
#include <stdio.h>

void ordenar(int v[], int n)
{
    for (int i = 0; i < n - 1; i++) {
        int pos_min = i;
        for (int j = i + 1; j < n; j++)
            if (v[j] < v[pos_min]) pos_min = j;
        int aux = v[i];
        v[i] = v[pos_min];
        v[pos_min] = aux;
    }
}

int main(void)
{
    int v[] = {29, 7, 45, 3, 12};
    ordenar(v, 5);
    for (int i = 0; i < 5; i++) printf("%d ", v[i]);
    printf("\n");
    return 0;
}
```

Ejemplo 19.6 · Criba de Eratóstenes (todos los primos hasta N: se tachan los múltiplos).

```c
#include <stdio.h>
#define N 50

int main(void)
{
    int es_primo[N + 1];
    for (int i = 0; i <= N; i++) es_primo[i] = 1;
    es_primo[0] = es_primo[1] = 0;

    for (int i = 2; i * i <= N; i++)
        if (es_primo[i])
            for (int j = i * i; j <= N; j += i)
                es_primo[j] = 0;

    for (int i = 2; i <= N; i++)
        if (es_primo[i]) printf("%d ", i);
    printf("\n");
    return 0;
}
```

Ejemplo 19.7 · Recursión: torres de Hanoi. Para mover n discos de A a C usando B: mueve n−1 de A a B, mueve el grande de A a C, mueve n−1 de B a C.

```c
#include <stdio.h>

void hanoi(int n, char origen, char destino, char aux)
{
    if (n == 0) return;                     // caso base
    hanoi(n - 1, origen, aux, destino);
    printf("Mover disco %d de %c a %c\n", n, origen, destino);
    hanoi(n - 1, aux, destino, origen);
}

int main(void)
{
    hanoi(3, 'A', 'C', 'B');
    return 0;
}
```

Ejemplo 19.8 · Recursión: potencia y suma de cifras.

```c
#include <stdio.h>

int potencia(int b, int e)
{
    if (e == 0) return 1;
    return b * potencia(b, e - 1);
}

int suma_cifras(int n)
{
    if (n < 10) return n;
    return n % 10 + suma_cifras(n / 10);
}

int main(void)
{
    printf("2^8 = %d\n", potencia(2, 8));
    printf("Suma de cifras de 4821 = %d\n", suma_cifras(4821));
    return 0;
}
```

✿ Capítulo 20 · Depuración y buenas prácticas

Leer los errores del compilador. Te dice archivo, línea y problema. Arregla **el primero** y recompila: los demás suelen ser consecuencia.

- ✿ *expected ';' before ...* → falta un `;` en la línea anterior.
- ✿ *undeclared identifier / implicit declaration* → variable o función sin declarar (¿falta un `#include` o un prototipo?).
- ✿ *undefined reference to 'sqrt'* → falta `-lm` al compilar.
- ✿ *warning: format '%d' expects ...* → el marcador de `printf`/`scanf` no coincide con el tipo.
- ✿ *Segmentation fault* (al ejecutar) → te saliste de un vector o usaste un puntero mal.

Depurar "a mano" con `printf`: pon `printf` en puntos clave para ver qué valen las variables:

```
printf("DEBUG: i=%d suma=%d\n", i, suma);
```

Depurar con `gdb` (herramienta de verdad). Compila con `-g` y ejecuta:

```
gcc -g -Wall prog.c -o prog
gdb ./prog
(gdb) break main     # parar al empezar
(gdb) run            # ejecutar
(gdb) next           # siguiente línea
(gdb) print suma     # ver una variable
(gdb) quit
```

Detectar fallos de memoria: `valgrind ./prog` (en Linux/WSL) avisa de fugas y accesos fuera de rango.

Buenas prácticas:

- ✿ **Indenta** el código (cada bloque un nivel más a la derecha). Se lee 10 veces mejor.
- ✿ **Nombres claros** (`suma_notas`, no `s2`). **Comenta el porqué**, no lo obvio.
- ✿ **Una función = una tarea.** Si una función hace de todo, divídela.
- ✿ **Inicializa** siempre las variables.
- ✿ **Compila siempre con `-Wall -Wextra`** y arregla los avisos.
- ✿ **Prueba casos límite:** 0, negativos, vector de 1 elemento, texto vacío, el valor más grande.
- ✿ Programa **poco a poco**: escribe 5–10 líneas, compila, prueba. Nunca 200 líneas de golpe.

Ejemplo 20.1 · Proyecto final: gestor de notas con menú, estructuras y ficheros. Junta casi todo lo aprendido: `struct`, vectores, funciones, `switch`, bucles y ficheros.

```c
#include <stdio.h>
#include <string.h>
#define MAX 50

typedef struct {
    char nombre[30];
    double nota;
} Alumno;

int leer_alumnos(Alumno lista[])
{
    int n;
    printf("Cuantos alumnos? ");
    scanf("%d", &n);
    if (n > MAX) n = MAX;
    for (int i = 0; i < n; i++) {
        printf("Nombre %d: ", i + 1);
        scanf("%29s", lista[i].nombre);
        printf("Nota: ");
        scanf("%lf", &lista[i].nota);
    }
    return n;
}

void mostrar(Alumno lista[], int n)
{
    for (int i = 0; i < n; i++)
        printf("%-15s %5.2f\n", lista[i].nombre, lista[i].nota);
}

double media(Alumno lista[], int n)
{
    double s = 0;
    for (int i = 0; i < n; i++) s += lista[i].nota;
    return n > 0 ? s / n : 0;
}

void guardar(Alumno lista[], int n)
{
    FILE *f = fopen("alumnos.txt", "w");
    if (f == NULL) { printf("Error al guardar\n"); return; }
    for (int i = 0; i < n; i++)
        fprintf(f, "%s %.2f\n", lista[i].nombre, lista[i].nota);
    fclose(f);
    printf("Guardado en alumnos.txt\n");
}

int main(void)
{
    Alumno lista[MAX];
    int n = 0, opcion;
    do {
        printf("\n1. Introducir alumnos\n2. Mostrar\n3. Media\n");
        printf("4. Guardar en fichero\n0. Salir\nOpcion: ");
        scanf("%d", &opcion);
        switch (opcion) {
            case 1: n = leer_alumnos(lista); break;
            case 2: mostrar(lista, n); break;
            case 3: printf("Media = %.2f\n", media(lista, n)); break;
            case 4: guardar(lista, n); break;
            case 0: printf("Hasta luego!\n"); break;
            default: printf("Opcion no valida\n");
        }
    } while (opcion != 0);
```

```
    return 0;
}
```

✿ Chuleta final: lo esencial en una página

Quiero…	Escribo…
Programa base	`#include <stdio.h>` + `int main(void){ ... return 0; }`
Compilar	`gcc -Wall prog.c -o prog -lm` y `./prog`
Leer int / double / char	`scanf("%d",&n)` · `scanf("%lf",&x)` · `scanf(" %c",&c)`
Escribir	`printf("%d %.2f %c %s\n", n, x, c, texto)`
Resto / cociente	`a % b` · `a / b` (¡enteros!)
Decisión	`if (c) {...} else if (c2) {...} else {...}`
Varios valores exactos	`switch (v) { case 1: ...; break; default: ... }`
Repetir n veces	`for (int i = 0; i < n; i++) {...}`
Repetir mientras	`while (c) {...}` · `do {...} while (c);`
Función	`tipo nombre(tipo p) { ... return r; }`
Vector	`int v[N];` índices `0..N-1`
Matriz	`int m[F][C];` → `m[i][j]`
Aleatorio 1..6	`srand(time(NULL));` una vez + `1 + rand() % 6`
Puntero	`int *p = &a;` · `*p` es el valor · `p` es la dirección
Función que modifica	`void f(int *x) { *x = 5; }` y se llama `f(&a);`
Comparar / copiar texto	`strcmp(a,b)==0` · `strcpy(dest,orig)`
Leer una línea	`fgets(cad, sizeof(cad), stdin);`
Reservar memoria	`int *v = malloc(n*sizeof(int));` … `free(v);`
Estructura	`typedef struct { int x; } T;` → `t.x` / `p->x`
Fichero	`FILE *f=fopen("a.txt","r");` … `fclose(f);`

♡ Los 18 errores que más se cometen

1. Olvidar el `;` o una `}`.
2. `=` en vez de `==` dentro de un `if`.

3. Olvidar el `&` en `scanf` (menos en cadenas).
4. `%f` en vez de `%lf` al leer un `double`.
5. `scanf("%c")` sin espacio antes (se salta la lectura).
6. División entera: `7/2` es 3.
7. Usar una variable sin inicializar.
8. Olvidar el `break` en un `switch`.
9. `;` justo después de `if (...)`, `for (...)` o `while (...)`.
10. Bucle infinito (no cambias la variable de la condición).
11. Salirse de un vector (`v[N]`) o recorrerlo con `<=`.
12. Llamar a una función antes de declararla (falta el prototipo).
13. Usar un puntero sin inicializar o con `NULL`.
14. Olvidar `free` (fuga de memoria) o usar la memoria después de `free`.
15. Comparar cadenas con `==` en vez de `strcmp`.
16. No comprobar que `fopen` o `malloc` no devolvieron `NULL`.
17. Una cadena sin hueco para el `'\0'` (`char s[4]` para "hola").
18. Olvidar `fclose` o abrir con `"w"` un fichero que querías conservar.

✿ Ejercicios propuestos (para practicar)

Hazlos en este orden: cada bloque usa lo del anterior. Te dejo una **pista** y, si te atascas, mándame tu intento y lo vemos juntas ♡

Secuenciales

1. Pide el precio de un producto y el % de IVA; muestra el precio final. *Pista:* `precio * (1 + iva / 100)`.
2. Pide una cantidad de euros y muestra cuántos billetes de 20, 10 y 5 y cuántas monedas de 1 € hacen falta. *Pista:* `/` y `%` encadenados, como los segundos del Ejemplo 6.4.
3. Pide los catetos de un triángulo rectángulo y calcula la hipotenusa. *Pista:* `sqrt` y `math.h`.

Selectivas

1. Pide un número y di si es positivo, negativo o cero.
2. Pide tres lados y di si forman triángulo (cada lado menor que la suma de los otros dos) y de qué tipo es (equilátero, isósceles, escaleno).
3. Programa una calculadora de notas: pide una nota y muestra la calificación con `else if` (suspenso, aprobado, notable, sobresaliente) y, con `switch`, el día de la semana a partir de un número 1–7.

Bucles

1. Muestra los números pares del 1 al 100. *Pista:* `for` con `i += 2`, o con `%`.
2. Pide un número y muestra la suma de sus cifras. *Pista:* `n % 10` y `n /= 10` dentro de un `while`.
3. Dibuja una pirámide de asteriscos con n filas. *Pista:* en cada fila, primero espacios y luego asteriscos (bucles anidados).
4. Pide números hasta que el usuario escriba un negativo y muestra el máximo.

Funciones

1. Escribe `int max(int a, int b)`, `int es_par(int n)` y `long long potencia(int b, int e)` y pruébalas desde `main`.
2. Escribe `int suma_cifras(int n)` y reutilízala en un programa que diga cuántos números del 1 al 1000 tienen suma de cifras 10.

Vectores y matrices

1. Lee 10 números, calcula su media y cuenta cuántos están por encima de ella.
2. Rellena un vector de 20 elementos con aleatorios 1–100, muéstralo ordenado de menor a mayor (burbuja) y calcula la mediana.
3. Crea una matriz 4×4 aleatoria y calcula la suma de su diagonal principal (`m[i][i]`) y la de cada fila.

4. Dado un texto, cuenta cuántas vocales tiene (cadenas + `switch`).

Punteros y memoria dinámica

1. Escribe `void ordenar2(int *a, int *b)` que deje en `a` el menor y en `b` el mayor.
2. Pide `n`, reserva un vector con `malloc`, rellénalo con aleatorios y muestra el máximo y el mínimo con una sola función (Ejemplo 13.4). No olvides el `free`.

Cadenas, estructuras y ficheros

1. Lee una frase con `fgets` y muéstrala al revés.
2. Define una estructura `Libro` (título, autor, páginas) y un vector de 5 libros; muestra el de más páginas.
3. Guarda en un fichero los 10 primeros términos de Fibonacci y léelos después para sumarlos.
4. Amplía el gestor de notas (Ejemplo 20.1) para que **cargue** los alumnos del fichero al empezar y muestre el mejor y el peor.

Cuando termines algunos, pásame tu código (o una foto) y te lo reviso: dónde falla, cómo mejorarlo y por qué.